Capability Receipts for Auditable AI Agent Execution: A Design Science Study of Lattice

Lakshman Turlapati
Independent Researcher
Email: lakshmanturlapati@gmail.com

***Abstract*: Organizations increasingly deploy large language model systems that route work across model providers, tools, files, modalities, and agent loops. These systems produce operational traces, yet the traces are usually not sufficient for audit: they can be edited after the fact, they often omit why a model was selected, they do not bind inputs to outputs, and they rarely support offline replay by a party that was not present during execution. This paper presents Lattice, a design science artifact for auditable AI agent execution. Lattice introduces *capability receipts*: canonical, redacted, signed, and replayable records of AI runs that capture intent, policy, routing, artifacts, invariants, outputs, cost, latency, and verification metadata. The paper makes two contributions. First, it defines a capability-receipt framework for signed, replayable, and policy-enforced execution across model and agent workflows. Second, it develops an organizational auditability model showing how capability receipts strengthen governance, accountability, reproducibility, and control compared with ordinary logs, observability traces, and provider wrappers. The artifact combines deterministic routing, terminal policy tripwires, tamper-evident signing, content-addressed replay, cross-language conformance vectors, and chained multi-agent provenance. Evaluation is presented through design requirements, implementation evidence, conformance checks, replay scenarios, tripwire tests, and an audit workflow. The result is a practical control layer that turns auditability from an external after-the-fact activity into a built-in property of each AI run.**

***Index Terms*: AI governance, AI auditability, design science research, capability receipts, agent execution, digital accountability, replayable systems, software provenance**

I. Introduction

AI applications are moving from narrow text-generation tasks into operational systems that triage work, extract information from documents, call tools, route between model providers, and coordinate agent loops. In these systems, a single business task may include prompts, files, images, JSON, model-selected tool calls, provider fallbacks, policy constraints, and multiple child agents. This creates a governance problem. The organization needs to know not only what the system returned, but also what it was asked to do, which model path was chosen, which constraints were applied, whether a policy violation stopped execution, and whether the record can be verified later.

Most production stacks answer these questions with logs, traces, dashboards, or model-provider metadata. Those mechanisms are useful for operations, but they are weak audit evidence. A log line can be edited. A trace may document latency without proving the integrity of the input-output relationship. A provider wrapper may normalize APIs without recording a deterministic reason for model selection. An agent framework may show a tool sequence without producing a portable artifact that an external reviewer can verify with a public key. The result is a gap between responsible-AI principles and the operational evidence needed to demonstrate that a particular run complied with organizational policy.

The information systems literature has long treated IT artifacts as objects that can encode organizational intent and control [1], [2]. The AI governance literature similarly emphasizes accountability, transparency, human oversight, and operationalization of principles [3]–[5]. Yet a concrete design question remains underdeveloped: what should an AI-agent runtime emit so that a later reviewer can verify a run without trusting the operator's unstructured logs?

This paper addresses that question through Lattice, a TypeScript-first runtime and protocol artifact for auditable AI execution. Lattice is capability-first: a developer describes the job, provides artifacts, declares desired outputs, and sets policy constraints. The runtime routes the work using an inspectable deterministic policy, executes the run, evaluates output invariants, and emits a signed capability receipt. The receipt is canonicalized, redacted before signing, signed inside a Dead Simple Signing Envelope (DSSE), and paired with content-addressed artifacts for offline replay.

The research question is:

> How can an AI-agent runtime produce portable, verifiable audit evidence for individual model and agent runs while preserving deterministic policy enforcement and replayability?

The paper makes two explicit contributions.

1) **Capability-receipt framework.** The paper defines capability receipts as a design primitive for signed, replayable, policy-enforced AI execution. A receipt binds run intent, route selection, policy constraints, redaction, output hashes, and verification metadata into a tamper-evident artifact.
2) **Organizational auditability model.** The paper maps receipt mechanisms to organizational control needs: governance evidence, accountability for routing and tool use, reproducibility through offline replay, and control through terminal tripwires that fail closed instead of retrying unsafe outputs.

The novelty is not that digital signatures, canonical

JSON, software provenance, model cards, or runtime traces are individually new. The novelty is their integration into a runtime-level audit artifact for AI-agent execution: deterministic routing explains why a model was selected; policy tripwires make unsafe outputs terminal; DSSE signing makes the run record tamper-evident; content-addressed replay allows independent verification; and chained receipts represent multi-agent provenance.

II. Literature Review

A. Design Science Research in Information Systems

Design science research (DSR) studies purposeful artifacts that solve relevant organizational problems while contributing generalizable design knowledge [1], [6], [7]. The DSR tradition is appropriate here because the core problem is not only descriptive. Organizations need a buildable artifact that changes the evidence available after AI execution. Following Gregor and Hevner [2], the paper positions Lattice as both an instantiation and a source of design principles for auditability: make run intent explicit, make route choice deterministic, make unsafe outputs terminal, bind records cryptographically, and support replay by a party outside the live execution environment.

B. Responsible AI Governance and Accountability

Responsible AI guidelines converge on principles such as transparency, accountability, privacy, robustness, fairness, and human oversight [3], [8], [9]. The challenge is operationalization. Recent information systems work argues that AI governance requires structural, relational, and procedural practices rather than abstract principles alone [4]. Algorithmic accountability work also shows that audit claims require traceability and evidence about the decisions made during system operation [10]–[12]. Lattice contributes at this operational layer. It does not replace governance policy; it provides a verifiable record that a governance process can inspect.

C. Algorithmic and Internal Auditing

Auditing research emphasizes evidence, control, independence, and the risk that organizational systems produce reports that cannot be independently verified [13]. In AI systems, the accountability gap is amplified by opaque models, evolving prompts, nondeterministic outputs, and external providers. Raji et al. [5] propose internal algorithmic auditing as an end-to-end accountability process. Lattice addresses a complementary technical question: what runtime evidence should such a process consume? A capability receipt is designed to be a stable audit object rather than a dashboard view or mutable operational trace.

D. Software Provenance and Reproducibility

The software supply-chain community has developed mature mechanisms for signed provenance. in-toto signs the steps that produce software artifacts [14]; DSSE defines an envelope for unambiguous payload signing [15]; SLSA defines supply-chain integrity levels [16]; and Sigstore makes software signing more accessible [17]. Lattice adapts this lineage from build artifacts to runtime AI work. The signed object is not a compiled binary; it is a single AI run. Reproducibility literature in machine learning similarly emphasizes that claims should be checkable and repeatable [18], [19]. Lattice narrows this to run-level replay: it does not claim to reproduce a provider's future nondeterministic behavior, but it verifies that the recorded run, artifacts, and output hash match the signed receipt.

E. Documentation, Transparency, and Observability

Model Cards and Datasheets for Datasets improve disclosure about models and datasets [20], [21]. Observability systems and agent frameworks expose traces, spans, sessions, and tool calls. Provider wrappers and agent frameworks such as the Vercel AI SDK, LangChain/LangGraph, LiteLLM, the OpenAI Agents SDK, and the Model Context Protocol normalize development interfaces [22]–[26]. These systems are complementary, but their core artifact is not a signed, portable, replayable run receipt. The gap is therefore not visibility in the broad sense. It is verifiable audit evidence for a specific AI execution.

III. Design Science Method

The study follows a DSR process with five activities adapted from established DSR guidance [1], [2], [7]. First, the problem is identified as insufficient audit evidence for AI-agent execution. Second, objectives are derived from governance and provenance needs. Third, the artifact is designed and implemented as Lattice. Fourth, the artifact is evaluated through executable invariants, conformance vectors, replay scenarios, and audit workflows. Fifth, the artifact and design knowledge are communicated through the manuscript and accompanying protocol documentation.

The artifact is evaluated as an operational control layer, not as a model accuracy benchmark. This distinction matters. The relevant question is whether the runtime produces evidence that can be verified, replayed, and used in governance review. Latency and model quality depend on external providers and are outside the receipt trust boundary.

Table I summarizes the design requirements and the mechanisms that implement them. These requirements are intended to be reusable design knowledge for systems that need auditability even if they do not adopt the Lattice implementation.

IV. Artifact Design

A. Capability-First Run Model

Lattice begins with an intent rather than a provider call. A run declares a task, artifacts, desired outputs, schemas, policies, and optional contracts. Artifacts can include text, JSON, images, documents, audio, or tool results. Policies express provider constraints, privacy classes, latency preferences, and budget. This design shifts the unit of control from "call this model" to "perform this capability

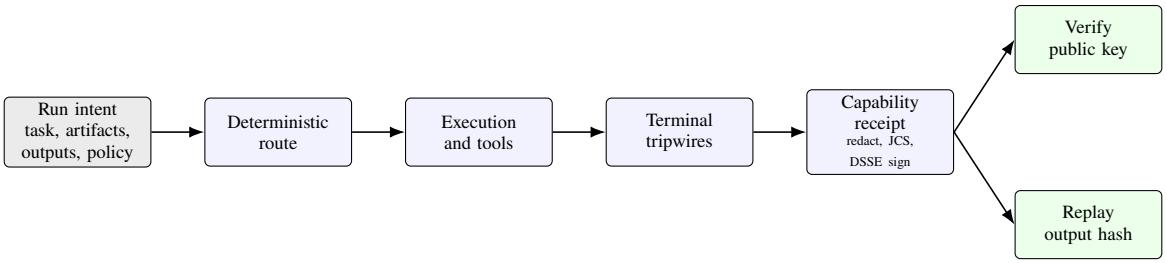

Fig. 1. Lattice audit evidence lifecycle. A capability-first run is routed by a deterministic decision function, executed under policy, stopped by terminal tripwires when needed, and committed to a signed receipt that can be verified and replayed offline.

TABLE I
DESIGN REQUIREMENTS FOR AUDITABLE AI-AGENT EXECUTION.

Requirement	Rationale	Lattice mechanism
DR1 Intent	Auditors need to know what the system was asked to do.	Capability-first run model
DR2 Routing	Model choice should be explainable and reproducible.	Pure routing function and execution plan
DR3 Enforcement	Unsafe outputs should fail closed.	Terminal tripwires and contracts
DR4 Integrity	Run records must be tamper-evident.	Redact, canonicalize, DSSE sign
DR5 Replay	Reviewers need offline verification.	Content-addressed replay envelope
DR6 Lineage	Multi-agent work needs parent-child accountability.	Chained receipts and lineage fields

under these constraints." The receipt can therefore record the organization's declared intent next to the runtime's actual route and outcome.

B. Deterministic and Inspectable Routing

Routing is implemented as a pure function over the capability catalog, request, and policy. It performs hard filters, policy filters, stable scoring, and deterministic tie breaking. The route decision records accepted candidates, rejected candidates, fallback options, and typed reasons when no route is possible. This makes model choice reviewable. An auditor can inspect why a provider was selected and whether alternatives were rejected for capability, privacy, budget, or policy reasons.

C. Policy Tripwires with Terminal Semantics

Output invariants act as tripwires. Examples include requirements to cite an artifact, avoid personally identifiable information, or match a declared schema. The important design choice is terminal semantics: when a tripwire fires, the fallback chain is forbidden from retrying the run. This prevents a common unsafe pattern in which a system keeps trying until a policy-violating output happens to pass. For auditability, the receipt records the terminal outcome rather than hiding it behind a later successful retry.

D. Capability Receipts

The capability receipt is the central artifact. A receipt is assembled from the run record, redacted, canonicalized using the JSON Canonicalization Scheme [27], and signed using Ed25519 [28] inside a DSSE envelope [15]. The ordering is deliberate: redaction occurs before signing so that secrets and detected personal information do not enter the signed bytes. The receipt includes the run identifier, route, model class where available, policy metadata, output hash, usage, cost, latency, redaction manifest, and key identifier. Verification rejects unsupported or downgraded schema versions before cryptographic work, reducing the risk that an older receipt shape can bypass newer integrity commitments.

E. Offline Replay and Content Addressing

Receipts become stronger audit evidence when they can be replayed. Lattice pairs receipts with content-addressed artifacts and materializes a replay envelope only after verifying the receipt. This verify-before-load ordering prevents a tampered receipt from triggering artifact loads or downstream side effects. Replay reconstructs the recorded run and recomputes the output hash. It does not claim to reproduce a provider's future model behavior; it verifies the integrity of the recorded execution.

F. Chained Multi-Agent Provenance

Agent workflows add another audit problem: a parent agent may delegate work to child agents, each of which calls tools or models. Lattice represents this with chained receipts. Child receipts can include a parent receipt content identifier and lineage metadata, allowing a reviewer to reconstruct a tree of delegated work. This is the agent-execution analogue of provenance chains in software supply-chain systems.

V. ORGANIZATIONAL AUDITABILITY MODEL

The organizational value of capability receipts can be summarized as four forms of auditability.

1) **Governance auditability.** A receipt records the policy and contract context that governed a run. This helps a governance team evaluate whether a run was performed under approved constraints.
2) **Accountability auditability.** A receipt records route selection, tool use, parent-child receipt links, and typed

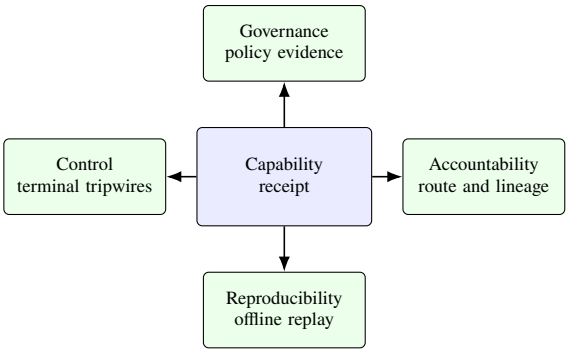

Fig. 2. Organizational auditability model. A receipt is the audit object that connects technical execution evidence to governance, accountability, reproducibility, and control review.

TABLE II
CAPABILITY RECEIPTS COMPARED WITH COMMON EVIDENCE MECHANISMS.

Mechanism	Strength	Audit limitation
Application logs	Easy to collect and search	Mutable and rarely bind inputs to outputs
Observability traces	Strong operational visibility	Usually not portable signed evidence
Provider metadata	Useful billing and usage facts	Provider-specific and incomplete for policy
Agent transcripts	Explain step sequences	May omit route rationale and integrity checks
Capability receipts	Signed, replayable, policy-aware run record	Requires key management and artifact retention

failure reasons. This creates an accountability chain for model and agent decisions.

3) **Reproducibility auditability.** A receipt and its content-addressed artifacts support offline replay and output-hash comparison.
4) **Control auditability.** Terminal tripwires and verification errors make policy enforcement visible as an auditable event rather than an internal implementation detail.

Table II contrasts this model with common alternatives.

This model does not remove the need for organizational governance. It gives the governance process a better object to inspect. A receipt can be attached to an incident, reviewed during a control assessment, used as a regression fixture, or sampled by an internal audit team.

A. Illustrative Use Case: Aircraft Component Warranty Adjudication

Aircraft-component warranty adjudication is a useful boundary case because the evidence and decision authority are distributed across organizations. IATA reports that warranty agreements and claim handling vary among original equipment manufacturers (OEMs), maintenance suppliers, and airlines; its guidance identifies part and serial numbers, time since new or overhaul, removal reason and date, aircraft registration, and sometimes photographs as core claim evidence [29]. More recently, IATA described maintenance data as fragmented across airlines, maintenance, repair, and overhaul organizations (MROs), OEMs, suppliers, and service providers, and linked warranty recovery to connecting maintenance events, component performance, repair history, and contractual terms [30]. ICAO's electronic aircraft maintenance records work foregrounds record integrity and transferability, while FAA Advisory Circular 43-9D provides active guidance on maintenance record-making and recordkeeping under 14 CFR parts 43 and 91 [31], [32].

Consider, illustratively, an airline seeking recovery after the premature removal of a serialized component. Its evidence set may contain part and serial data, removal and installation records, aircraft telemetry, borescope images or video, technician notes and voice material, prior repair history, maintenance manual excerpts, and applicable warranty terms. These artifacts differ in format, sensitivity, custody, and contractual relevance. Copying all of them into a general-purpose model endpoint would create an unnecessary disclosure surface and would still leave the resulting recommendation difficult to audit.

The airline or MRO first performs a *restricted local run*. The capability catalog exposes only an approved local or on-premises provider for restricted data; a provider allow-list, `privacy: restricted`, `noUpload`, and `noPublicUrl` constraints make that boundary explicit. Deterministic routing rejects candidates that cannot satisfy it. The run does not decide the claim. It produces typed findings—for example, an event chronology, extracted part identifiers, measured values, and candidate failure observations—with lineage back to the source artifacts. Its execution plan and receipt record the selected route, rejected alternatives, transformations, input commitments, and output commitment.

Only the derived findings and necessary evidence slices proceed to a second, *contract-bound warranty run*, as shown in Figure 3. The contract binds the applicable warranty terms, an output schema, approved providers, privacy constraints, and a cost ceiling. Structured output can separate eligibility, supporting and contrary evidence, contractual basis, uncertainty, and a recommended commercial action. A `must-cite` tripwire requires evidence references, a `field-from-table` tripwire restricts specified fields to values admitted by the contractual table, and a `no-pii` tripwire prevents prohibited personal-data patterns detected at the configured result path from entering the shareable result. A violation is terminal: Lattice records the failed control and does not try another model until one happens to pass.

For a successful run, the operator assembles a portable claim packet containing the recommendation, execution plan, signed receipt, necessary artifacts, and replay evidence. An OEM, insurer, or lessor can use the public key and artifact/output hashes to verify the signer and commitments, inspect why the route was selected, and replay the recorded result without access to the operator's live system. The mapping to the four auditability dimensions is direct: policy and contract

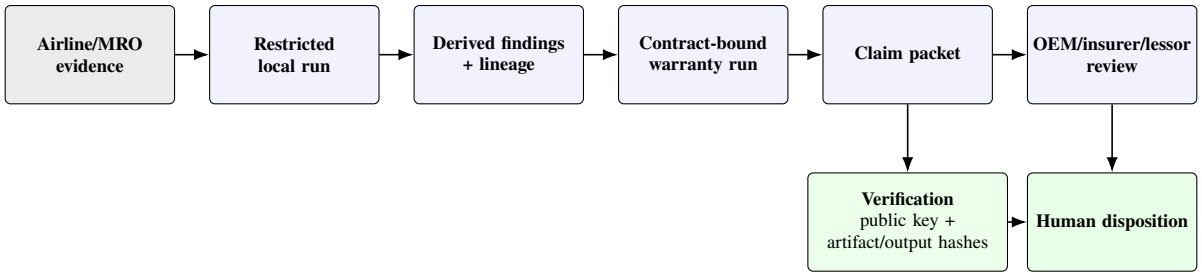

Fig. 3. Illustrative workflow for cross-organizational aircraft-component warranty adjudication. The verification branch checks portable execution evidence before human disposition. This is a design application, not a deployed evaluation.

commitments support *governance*; route rationale and artifact lineage support *accountability*; hashes and replay support *reproducibility*; and terminal tripwire verdicts support *control*. The packet informs review, but authorized engineering, maintenance, commercial, and legal personnel retain authority over the final warranty and maintenance disposition.

The assurance boundary is procedural. Lattice can prove that a signer bound a specified execution procedure and output commitment to particular artifact hashes, policy, route, and control verdict. It does not prove sensor accuracy, the truth of maintenance records, media authenticity, model correctness, regulatory compliance, or aircraft airworthiness. C2PA Content Credentials can complement Lattice by carrying capture and edit provenance for images or video, but C2PA likewise presents provenance assertions for human trust decisions rather than deciding their truth [33]. Neither mechanism substitutes for approved maintenance data, regulatory processes, or accountable human judgment.

VI. Evaluation

The evaluation asks whether the artifact satisfies the design requirements in Table I. The current implementation provides five classes of evidence.

A. Executable Invariants

Routing and tripwire evaluation are pure kernels. Tests assert deterministic outputs for fixed inputs and ensure that terminal violations are not retried by fallback logic. The design requirement tested here is that policy enforcement is not merely logged; it changes execution control flow.

B. Receipt Verification

The verifier checks envelope shape, schema version, key existence and state, canonicalization, and signature validity. Negative tests cover malformed envelopes, unsupported versions, revoked keys, canonicalization mismatches, and invalid signatures. These tests support the integrity requirement: a changed receipt should fail verification.

C. Replay Scenarios

Replay tests assert that artifact loading does not occur until receipt verification succeeds. They also assert that materialization and offline replay preserve the output hash. These scenarios evaluate the reproducibility requirement: a reviewer can compare the recorded output commitment with a recomputed value.

D. Cross-Language Conformance

The repository includes a language-neutral receipt specification, JSON schemas for accepted receipt versions, conformance vectors, a TypeScript verification harness, and a Python reference client. Positive vectors cover accepted schema versions, while negative vectors cover the verification error taxonomy. The Python client verifies and mints receipts against the same vector set. This evidence supports portability: the receipt is a protocol artifact, not only a TypeScript implementation detail.

E. Audit Workflow Demonstration

The command-line workflow turns receipts into review objects. A reviewer can verify a receipt, materialize a replay envelope, compare output hashes, and use a directory of receipts as a regression baseline. The workflow demonstrates how capability receipts can move from runtime instrumentation into a practical governance process.

VII. Discussion

Capability receipts operationalize AI governance at the run level. Responsible AI frameworks often ask whether a system is accountable, transparent, or auditable. Lattice makes those questions concrete: for this run, what was the intent, which route was selected, which policy was enforced, what output was produced, which key signed the record, and can the output commitment be checked later?

The design also clarifies the difference between observability and auditability. Observability helps engineers understand a live system. Auditability requires evidence that can survive outside the live system and be checked independently. Signed capability receipts are therefore not a replacement for traces; they are a narrower and stronger evidence layer.

There are tradeoffs. Redaction protects secrets but can reduce replay fidelity when a secret influenced the output. Deterministic routing improves reviewability but limits opaque model-selected routing in the initial design. Provider parity keeps receipts portable but may defer provider-specific features. These tradeoffs are appropriate for the artifact's

governance purpose: a control layer should prefer clear, reviewable behavior over hidden convenience.

VIII. Limitations and Future Work

The paper has several limitations. First, the evaluation is artifact-centered. It shows that the implemented mechanisms satisfy design requirements, but it does not yet include field evidence from auditors, compliance teams, or governance boards. Second, replay verifies the recorded run and output hash; it does not reproduce future nondeterministic model behavior. Third, exact cross-language object-output hashing remains outside the current conformance boundary because JSON serializers differ across languages. Fourth, the artifact assumes key management discipline. A signed receipt is only as strong as the organization's key lifecycle, revocation process, and artifact retention policy.

Future work should evaluate the artifact in organizational audit settings, develop control checklists for receipt review, expand language clients, add more adversarial conformance vectors, and study how receipt sampling can support ongoing AI governance. A second research path is comparative: evaluate whether receipt-based audit workflows improve reviewer confidence, defect discovery, or policy compliance relative to log-only and trace-only workflows.

IX. Conclusion

This paper presented Lattice as a design science artifact for auditable AI-agent execution. The first contribution is a capability-receipt framework: a signed, replayable, policy-enforced run record that binds intent, route, constraints, artifacts, outputs, and verification metadata. The second contribution is an organizational auditability model: capability receipts improve governance, accountability, reproducibility, and control by providing stronger evidence than ordinary logs, traces, and provider wrappers. The implementation demonstrates the design through deterministic routing, terminal tripwires, tamper-evident DSSE signing, content-addressed replay, cross-language conformance, and chained multi-agent provenance. The broader implication is that auditability can be designed into the runtime itself rather than reconstructed after the fact.

References

[1] A. R. Hevner, S. T. March, J. Park, and S. Ram, "Design science in information systems research," *MIS Quarterly*, vol. 28, no. 1, pp. 75–105, 2004.

[2] S. Gregor and A. R. Hevner, "Positioning and presenting design science research for maximum impact," *MIS Quarterly*, vol. 37, no. 2, pp. 337–355, 2013.

[3] A. Jobin, M. Ienca, and E. Vayena, "The global landscape of AI ethics guidelines," *Nature Machine Intelligence*, vol. 1, pp. 389–399, 2019.

[4] N. Berente, B. Gu, J. Recker, and R. Santhanam, "Managing artificial intelligence," *MIS Quarterly*, vol. 45, no. 3, pp. 1433–1450, 2021.

[5] I. D. Raji, A. Smart, R. N. White, M. Mitchell, T. Gebru, B. Hutchinson, J. Smith-Loud, D. Theron, and P. Barnes, "Closing the AI accountability gap: Defining an end-to-end framework for internal algorithmic auditing," in *Proceedings of the 2020 Conference on Fairness, Accountability, and Transparency*. ACM, 2020, pp. 33–44.

[6] S. T. March and G. F. Smith, "Design and natural science research on information technology," *Decision Support Systems*, vol. 15, no. 4, pp. 251–266, 1995.

[7] K. Peffers, T. Tuunanen, M. A. Rothenberger, and S. Chatterjee, "A design science research methodology for information systems research," *Journal of Management Information Systems*, vol. 24, no. 3, pp. 45–77, 2007.

[8] L. Floridi, J. Cowls, M. Beltrametti, R. Chatila, P. Chazerand, V. Dignum, C. Luetge, R. Madelin, U. Pagallo, F. Rossi, B. Schafer, P. Valcke, and E. Vayena, "AI4People: An ethical framework for a good AI society," *Minds and Machines*, vol. 28, no. 4, pp. 689–707, 2018.

[9] National Institute of Standards and Technology, "Artificial intelligence risk management framework (AI RMF 1.0)," NIST AI 100-1, 2023, https://www.nist.gov/itl/ai-risk-management-framework.

[10] N. Diakopoulos, "Accountability in algorithmic decision making," *Communications of the ACM*, vol. 59, no. 2, pp. 56–62, 2016.

[11] M. Wieringa, "What to account for when accounting for algorithms: A systematic literature review on algorithmic accountability," in *Proceedings of the 2020 Conference on Fairness, Accountability, and Transparency*. ACM, 2020, pp. 1–18.

[12] J. A. Kroll, "Outlining traceability: A principle for operationalizing accountability in computing systems," in *Proceedings of the 2021 ACM Conference on Fairness, Accountability, and Transparency*. ACM, 2021, pp. 758–771.

[13] M. Power, *The Audit Society: Rituals of Verification*. Oxford University Press, 1997.

[14] S. Torres-Arias, H. Afzali, T. K. Kuppusamy, R. Curtmola, and J. Cappos, "in-toto: Providing farm-to-table guarantees for bits and bytes," in *Proceedings of the 28th USENIX Security Symposium (USENIX Security)*. USENIX Association, 2019.

[15] Secure Systems Lab and the in-toto project, "Dead simple signing envelope (DSSE) specification," Online specification, v1.0, 2021, https://github.com/secure-systems-lab/dsse.

[16] Open Source Security Foundation (OpenSSF), "Supply-chain levels for software artifacts (SLSA) framework," Online specification, 2023, https://slsa.dev.

[17] Z. Newman, J. S. Meyers, and S. Torres-Arias, "Sigstore: Software signing for everybody," in *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security (CCS)*. ACM, 2022.

[18] J. Pineau, P. Vincent-Lamarre, K. Sinha, V. Larivière, A. Beygelzimer, F. d'Alché Buc, E. Fox, and H. Larochelle, "Improving reproducibility in machine learning research (a report from the NeurIPS 2019 reproducibility program)," *Journal of Machine Learning Research*, vol. 22, no. 164, pp. 1–20, 2021.

[19] D. Sculley, G. Holt, D. Golovin, E. Davydov, T. Phillips, D. Ebner, V. Chaudhary, M. Young, J.-F. Crespo, and D. Dennison, "Hidden technical debt in machine learning systems," in *Advances in Neural Information Processing Systems*, vol. 28, 2015.

[20] M. Mitchell, S. Wu, A. Zaldivar, P. Barnes, L. Vasserman, B. Hutchinson, E. Spitzer, I. D. Raji, and T. Gebru, "Model cards for model reporting," in *Proceedings of the Conference on Fairness, Accountability, and Transparency (FAccT)*. ACM, 2019.

[21] T. Gebru, J. Morgenstern, B. Vecchione, J. W. Vaughan, H. Wallach, H. D. III, and K. Crawford, "Datasheets for datasets," *Communications of the ACM*, vol. 64, no. 12, pp. 86–92, 2021.

[22] Vercel, "AI SDK," Software, online documentation, 2024, https://sdk.vercel.ai.

[23] LangChain Inc., "LangChain and LangGraph," Software, online documentation, 2023, https://www.langchain.com.

[24] BerriAI, "LiteLLM: Call 100+ LLM APIs in the OpenAI format," Software, online documentation, 2023, https://github.com/BerriAI/litellm.

[25] OpenAI, "OpenAI agents SDK," Software, online documentation, 2024, https://github.com/openai/openai-agents-python.

[26] Anthropic, "Model context protocol (MCP)," Online specification, 2024, https://modelcontextprotocol.io.

[27] A. Rundgren, B. Jordan, and S. Erdtman, "JSON canonicalization scheme (JCS)," RFC 8785, Internet Engineering Task Force (IETF), 2020, https://www.rfc-editor.org/rfc/rfc8785.

[28] S. Josefsson and I. Liusvaara, "Edwards-curve digital signature algorithm (EdDSA)," RFC 8032, Internet Engineering Task Force (IETF), 2017, https://www.rfc-editor.org/rfc/rfc8032.

[29] IATA Maintenance Cost Technical Group, *Warranty Management Essentials*, 1st ed., International Air Transport Association, 2024, [Online]. Available: IATA source.

[30] S. Fox, “WMES 2026 speech,” International Air Transport Association, Jun. 2026, speech delivered June 24, 2026. [Online]. Available: IATA source.
[31] International Civil Aviation Organization, “Electronic aircraft maintenance records,” n.d., accessed: July 29, 2026. [Online]. Available: ICAO source.
[32] Federal Aviation Administration, “Maintenance records and FAA form 8130-3 Return to Service,” U.S. Department of Transportation, Advisory Circular 43-9D, Sep. 2025, issued September 22, 2025. [Online]. Available: FAA source.
[33] Coalition for Content Provenance and Authenticity, *Content Credentials Technical Specification 2.4*, Apr. 2026, [Online]. Available: C2PA source.